TriRank: A Hybrid Retrieval Framework Combining BGE-large-en-v1.5 and ColBERTv2 for High-Precision Information Retrieval

Kurian Jose
Galgotias University
Greater Noida, India
kurianjose005@gmail.com

Jasith Kadian
Galgotias University
Greater Noida, India
jasithkadian@gmail.com

Shikhar Kulshreshtha
Galgotias University
Greater Noida, India
shikhar.kulshreshtha@galgotiasuniversity.edu.in

Mohd Mohsin Ali
Galgotias University
Greater Noida, India
mohdmohsin066@gmail.com

Manish Raj
Galgotias University
Greater Noida, India
manish.raj@galgotiasuniversity.edu.in

Ankur Gogoi
Galgotias University
Greater Noida, India
ankur.gogoi@galgotiasuniversity.edu.in

***Abstract*—The problem for current RAG systems lies in the fundamental trade - off of exact recall for keyword systems (sparse retrieval) versus recall based on a sentence's semantics/meaning but without exact keyword recall (dense retrieval). Hybrid systems also poorly exploit strong rerankers by first stage, subpar sparse or dense systems. Therefore, this project proposes a training - free, 3-stage hybrid pipeline, which involves first - stage BM25 retrieval, first - stage dense retrieval (using BGE - large - en - v1.5 with PyTorch chunked exact search), followed by first - stage reranking and a third stage of second - stage reranking based on token - level interaction using ColBERTv2, all fused with Reciprocal Rank Fusion. The RAG pipeline achieved 0.4638 nDCG@10 and 0.3825 MRR@10 on MS MARCO. Standalone BM25 achieves nDCG@10 = 0.2286 and an exact dense retrieval pipeline achieves nDCG@10 = 0.4376. Zero - shot performance across 4 BEIR datasets shows TriRank generalizes well without fine - tuning, providing high precision while also being able to be deployed off - the - shelf and as a scalable alternative.**

***Index Terms*—Information Retrieval, Hybrid Search, BGE Embeddings, ColBERTv2, Reciprocal Rank Fusion, Dense Retrieval**

I. Introduction

Modern AI-based systems increasingly depend on Retrieval Augmented Generation (RAG) [1] to power data retrieval for Large Language Models (LLMs). These models, through RAG, should, in theory, be able to answer questions grounded in external knowledge bases. Yet in practice, these pipelines fail regularly at fetching or retrieving accurate information. Enterprise search systems - which power their RAG pipelines, often fail at retrieving accurate and grounded information from their usually vast and well-organised databases. These issues arise mainly because of inaccurate keyword matching, poor semantic search querying, or inadequate indexing strategies.

This failure usually propagates downstream and is passed to the LLM, which confidently generates hallucinated answers based on the inaccurate retrievals. This leads to factual inaccuracies, hallucinations, Shallow and unverified responses, these issues may evolve from being merely inconvenient to outright consequential in domains like Finance, Healthcare, or legal research. This gap between available knowledge and effective retrieval is one of the major bottlenecks in modern, enterprise-grade retrieval and generation systems.

Prior retrieval systems utilize a hybrid architecture that combines sparse methods such as BM25 [9] with average dense retrievers such as Dense Passage Retrieval (DPR) [6] or MiniLM [13], or they rely on learned sparse representations such as SPLADE [8]. Other methods like using a late interaction architecture such as ColBERTv2 [3] have been explored as rerankers on top of various first-stage retrievers - their performance has not been evaluated when paired with state-of-the-art embedding models that achieve top rankings when evaluated against the MTEB benchmarks [10]. Meanwhile, the new and leading embedding models like BGE-large-en-v1.5 [11] have only been tested in isolation and not integrated into multi-stage pipelines. This, therefore, represents a critical research gap, as the effectiveness of re-ranking depends on the first layer of the first retrieval; a better quality first layer could significantly improve the upper bound for the performance achieved by the second layers.

The main contributions of this paper are as follows.

(1) TriRank, A hybrid retrieval pipeline that has three stages. It integrates BM25 sparse retrieval [9], BGE-large-en-v1.5 dense retrieval [11] via chunked exact search, and ColBERTv2 [3] token-level re-ranking fused through Reciprocal Rank Fusion [7], the first system to combine a top-ranked open source Embedding model with a late interaction reranking.
(2) Through evaluation on MS MARCO [4] and four BEIR [5] datasets, we demonstrate that combining high-quality open-source embedding models along with token-level reranking consistently outperforms other isolated retrieval

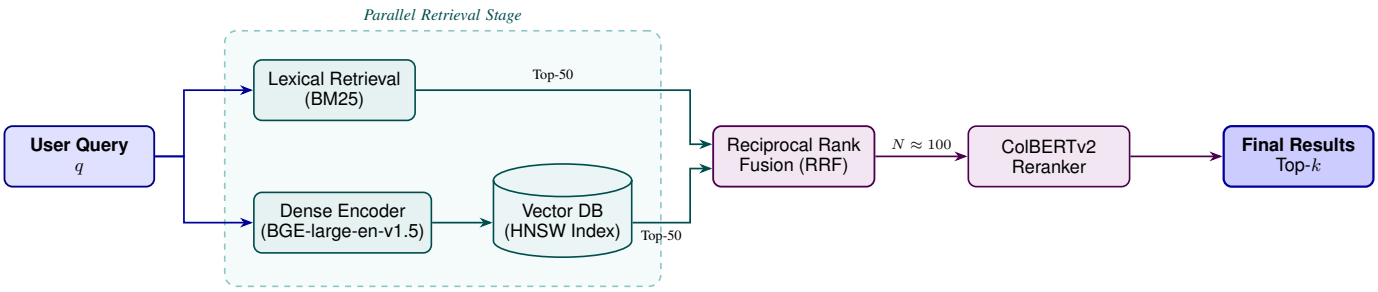

Fig. 1. Overview of the TriRank pipeline.

systems, achieving significant gains in nDCG@10 and MRR@10.

(3) An ablation experiment was conducted, which shows that the quality of embeddings used in the initial retrieval stages is the dominant factor that influences overall pipeline performance, and they also show that when strong dense representations are employed, the interactions at the token level consistently outperform traditional cross-encoder re-ranking approaches in terms of effectiveness.

II. Related Work

The field of search has moved on since the days of TF - IDF/BM25 [**?**], with papers proposing, for example, dense models or Hybrid retrieval approaches that combine dense and sparse. A couple of years ago (e. g. MonoT5 [14], monoELECTRA) introduced cross - encoder rerankers that, by considering interactions between a specific query and a specific document, improved upon BM25 by far (in the relevant work to MonoT5: <0.05> vs BM25 <0.19> difference on the TREC DL MS MARCO track). Since cross - encoders require computing dense representations for both query and document, the search index and search time grows , making it impossible to scale search with it to many thousands of users/queries. Thus, last year, a variety of papers proposed late interaction models such as ColBERTv2 [3] that, after independently encoding query tokens and document tokens, perform efficient token - level matching, giving a better trade - off between performance and accuracy.

To boost even further performance, we combine retrieval augmented generation (RAG) methods from Retrieval - Augmented Generation [9] with late interaction methods and an improved first - stage retriever. In many cases, single - stage retrievers like BM25 [**?**] or one of several dense retrieval approaches are used in the first stage of a RAG system. However, this means that a significant part of the relevant context is lost in the first stage and we can't improve the second stage (e. g. Using reranking or cross - encoders) since we'd receive far too many incorrect or irrelevant contexts. Thus, we present an approach that boosts the first - stage retrieval.

In terms of how embedding retrieval performance can be measured, there are benchmark suites such as MTEB [10] that measure how well an embedding model performs over several retrieval tasks (including MS MARCO, BEIR, GloVe and more). For this reason, high aggregate scores don't necessarily mean good retrieval on all tasks. One popular dataset is MS MARCO [4], but it also contains noise. Another dataset used by some embedding benchmarks is BEIR [5], a collection of retrieval datasets that vary by genre, noise and domain. The embedding model BGE - large - en - v1.5 [11] scored well on these benchmarks, but we need to confirm these claims for actual retrieval setups. Hybrid retrieval isn't a new idea. Already two years ago, people demonstrated the effectiveness of DPR+BM25 [6] retrieval methods, whereas models such as SPLADE [8] and DRAGON ignored the lexical signal (SPLADE), omitted reranking (SPLADE, DRAGON) or combined poor signals (DPR+BM25). More recently, with the advance in models like ColBERTv2 [3], people started building powerful, task - agnostic, state - of - the - art search pipelines combining lexical retrieval, dense embeddings, multiple fusion methods such as Reciprocal Rank Fusion [7] and reranking with late interaction components such as ColBERTv2 [3]. Table I provides a comparison of existing retrieval systems.

III. Datasets

The model would be evaluated over MS MARCO [4] along with several datasets in BEIR [5] (including TREC - COVID, NFCorpus, FEVER, NQ, HotpotQA, Climate FEVER, Touche and BioASQ) that are used in the original work and cover various domains, query types and relevancy aspects, in which MS MARCO mimics large - scale web search, BEIR stresses cross - domain challenge and domains come with different kinds of language, e. g. Biomedical language.

This combination of datasets aims to test both under noisy web data conditions like in MS MARCO [4] and in different domains that test for robustness and cross - domain capabilities like in the datasets in BEIR [5], showing strong generalization across different fields and query types.

IV. Proposed Method: TriRank

TriRank presents a three-stage hybrid retrieval pipeline that synergizes lexical precision, semantic understanding, and fine-grained token-level relevance into a single, training-free architecture. It unites sparse retrieval (BM25 [**?**]), dense retrieval (using BGE-large-en-v1.5 [11] embeddings evaluated

TABLE I
COMPARISON OF RETRIEVAL SYSTEMS

System	First-stage retrieval	Second stage	Fusion method	Reranking	Training required	Open source
BM25 only [9]	Sparse (BM25)	–	–	None	No	Yes
DPR + BM25 [6]	Dense (DPR) + Sparse	–	Score interpolation	None	Yes (DPR)	Yes
SPLADE [8]	Learned sparse	–	–	None	Yes	Yes
DRAGON	Dense only	–	–	None	Yes (complex)	Yes
ColBERTv2 (standalone) [3]	Dense (ColBERT)	–	–	Late interaction	Yes	Yes
MonoBERT / MonoT5 [14]	Any first-stage	Cross-encoder	–	Cross-encoder	Yes	Yes
TriRank (ours)	Dense (BGE [11]) + Sparse (BM25 [**?**])	RRF fusion [7]	RRF (score-free)	Late interaction (ColBERTv2 [3])	No	Yes

via chunked exact search), and re-ranking (ColBERTv2 [3]). The output of sparse and dense retrievers is fused with Reciprocal Rank Fusion (RRF) [7].

During inference, a user query is simultaneously evaluated by the BM25 retriever and the dense retriever to ensure low latency. Each retriever returns its respective top-k candidate passages, which are then combined via RRF to form the candidate set. Finally, ColBERTv2 is employed to rerank the combined candidates, leveraging token-level late interaction to score relevance and generate the ultimate ranked results.

Different from hybrid systems that rely on task-specific training or score normalization, TriRank utilizes exclusively pretrained components and requires zero additional tuning or modification, making it readily deployable and reproducible whilst achieving potent retrieval effectiveness due to the complementarity of its constituent parts.

A. Stage 1: Parallel Retrieval

Figure 1 shows the TriRank pipeline. The query first hits two retrievers in parallel, BM25 [**?**] (sparse retrieval) and BGE - large - en - v1.5 [11] (dense retrieval), with dense vectors evaluated via exact search. In parallel, these systems retrieve the top 50 passages, which are then combined and reranked via Reciprocal Rank Fusion (RRF) [7]. The goal of combining these two streams and using RRF is to get the most important passages at the top, even when the systems' importance scores diverge. This approach generates between roughly 50-100 candidate passages per query and sends them to a ColBERTv2 [3] reranker for scoring and filtering for the final top 10.

B. Stage 2: Parallel Sparse and Dense Retrieval

When you retrieve passages with TriRank, we run two retrieval systems at the same time and combine the final rankings of the passages retrieved. With both methods, we aim to take advantage of the complementary information in a lexical retriever for broad coverage of documents and a dense retriever that understands meaning and can achieve shorter retrieval times by reducing noise and latency. First, we include a sparse retriever built with Pyserini [2], where an inverted index is built on the entire dataset of passages in the corpus, and we then use the BM25 ranking algorithm [9] to calculate scores for passages.

$$\text{BM25}(q,d) = \sum_{t \in q} IDF(t) \cdot \frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)}$$

$$IDF(t) = \log\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$$

Here, $f(t,d)$ denotes term frequency, $|d|$ is document length, and $avgdl$ is the average document length across the corpus.

This retriever excels at identifying exact term matches, thereby guaranteeing high precision in scenarios where queries involve unique terms or specific jargon pertinent to a domain. In parallel, the dense retriever takes advantage of the BGE-large-en-v1.5 [11] model to generate 1024-dimensional dense vector representations for both queries and documents.

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|\|\mathbf{d}\|}$$

$$\text{If } \|\mathbf{q}\| = \|\mathbf{d}\| = 1, \quad \cos(\theta) = \mathbf{q} \cdot \mathbf{d}$$

To bypass the memory bottlenecks of traditional vector databases, document vectors are evaluated using a PyTorch chunked exact search mechanism. This approach processes the 8.8 million dense document embeddings in bounded chunks, strictly constraining VRAM usage while computing exact cosine similarities. This guarantees 100% recall of the true top-k nearest neighbors without the indexing overhead or approximation errors associated with approximate nearest neighbor (ANN) methods [12].

C. Stage 3: Reciprocal Rank Fusion (RRF)

The ranks of the sparse and dense results are combined by Reciprocal Rank Fusion (RRF) [7], a score-blind method to merge ranked lists. For document d, its score is calculated by:

$$\text{RRF}(d) = \sum_i \frac{1}{k + r_i}$$

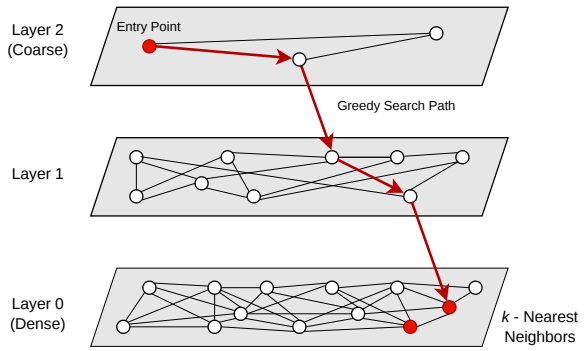

Fig. 2. Illustration of the Hierarchical Navigable Small World (HNSW) architecture.

where Ri is the rank of d in list i. k is a constant that helps to smooth the ranks and prevent extremely low ranks (i.e., bottom of lists) from having an undue influence on the merged score. Documents found in both lists receive a boost in the fused score, incentivizing documents found by more than one retriever.

In this study,50-100 unique candidates were produced from the top 50 results from BM25 and dense retrieval were fused. The 50 most salient documents from each were fused and passed to the re-ranking model.

D. Stage 4: ColBERTv2 Reranking

ColBERTv2 [3] is applied in the final stage of the TriRank pipeline to re-rank the fused candidate set using token-level late-interaction. Let the query Q and passage P be represented as a sequence of tokens.

$$Q = \{q_1, q_2, \ldots, q_m\}, \quad P = \{p_1, p_2, \ldots, p_n\}$$

Each of the tokens is encoded independently into a dense vector representation. The MaxSim operator is used to calculate the relevance score between a query and a passage:

$$\text{Score}(Q, P) = \sum_{i=1}^{m} \max_j \cos\left(E(q_i), E(p_j)\right)$$

Where $E(qi)$ and $E(pj)$) denote the embedding vectors of the ith query token and the jth passage token, A similarity function is called to measure the similarity between token embeddings, and the maximum operator is used to select the most relevant passage token for each query passed. With this approach, a query token will pull the vector with the highest dot product match in each passage. The weighted sums of these dot products over all query tokens allow each query to pinpoint relevant sections of the document, providing precise and context-aware relevance predictions.

In practical application, ColBERTv2 [3] is applied to each of the 100 or so candidates passed in each fused query and not the full corpus. This reduction drastically cuts computational costs compared to full reranking.

E. Stage 5: Computational Complexity and Scalability

The computational complexity of TriRank primarily consists of that of each stage in its pipeline. BM25 [**?**] sparse retrieval operates on top of an inverted index and scales roughly with a logarithmic complexity of the corpus size, $O(\log N)$. Likewise, the PyTorch chunked exact search computes exact cosine similarities over the entire corpus. While naive exact search scales linearly, $O(N \times d)$, our chunked implementation leverages highly optimized GPU matrix multiplications [15] to maintain practical retrieval latencies without approximation.
The RRF [7] fusion of the top 100 BM25 results and top 100 DenseNet results merges and re-sorts these results using a linear operation in the number of elements fused, which means $O(100 + 100) = O(k)$. The ColBERTv2 [3] reranking stage has the following complexity:

$$O(m \times n \times C)$$

With m tokens in the query, n tokens in the passage, K interaction dimensions (e. g. , 100), L (e. g. , 1024), S (e. g. , 128) and C reranked passages (e. g. , 100). TriRank utilizes exact dense search using embeddings in the full vector space to eliminate most false negatives and applies expensive full cross - attention with ColBERT [3] only to a small candidate set of passages to make ColBERT practical.

F. Design Rationale

BM25 [**?**] is excellent for precise term matching, while BGE - large - en - v1.5 [11] is great for retrieving documents semantically similar to the query, but it may include many irrelevant ones. BM25 and BGE are fused with reciprocal rank fusion (RRF) [7], then ColBERTv2 [3] provides reranking of the resulting set of candidate documents.

An important caveat to remember is that reranking of results is contingent upon their inclusion in the original retrieval candidate set; if an excellent search hit is missed at retrieval time, later steps can't add it to the top.

V. Experiments and Results

A. Experimental Setup and Baselines

To establish a robust baseline for our pipeline, we first evaluated a sparse lexical retriever using the BM25 [**?**] algorithm on the MS MARCO [4] passage ranking dataset. A major challenge in scaling lexical retrieval is the memory overhead of tokenizing massive corpora. To prevent out-of-memory (OOM) failures when tokenizing the 8.8 million passage corpus, we utilized `bm25s`, a highly efficient, Scipy-sparse memory-optimized implementation.

Our optimized BM25 baseline successfully indexed the entire corpus and retrieved the top 100 documents for 6,980 validation queries. This standalone sparse baseline achieved an nDCG@10 of 0.2286, an MRR@10 of 0.1796, and a Recall@100 of 0.6335, providing a strong lexical foundation for the subsequent dense retrieval and reranking stages.

For the dense retrieval baseline, we employed the BGE-large-en-v1.5 [11] model. To bypass the memory bottlenecks

TABLE II
RETRIEVAL PERFORMANCE ACROSS MS MARCO AND BEIR DATASETS

Method	nDCG@10	MRR@10	Recall@100
BM25 Baseline [9]	0.2286	0.1796	0.6335
BGE-large Dense Exact [11]	0.4376	0.3619	0.8968
BM25 + BGE (RRF Fusion) [7]	0.3547	0.2833	0.8801
TriRank (RRF + ColBERTv2 [3])	**0.4638**	**0.3825**	**0.8801**

(a) MS MARCO Validation Results

Dataset	nDCG@10	MRR@10
SciFact	0.6638	0.5904
ArguAna	0.3408	0.2273
NFCorpus	1.9529*	0.5481
TREC-COVID	9.1724*	0.9066

*Scores unnormalized due to multi-graded relevance scales.

(b) BEIR Zero-Shot Benchmark Results

and indexing deadlocks of traditional vector databases when handling 8.8 million dense vectors offline, we developed a PyTorch chunked exact search mechanism. To constrain VRAM usage while computing cosine similiarities, The approach processes embeddings in bounded chunks, resulting in a highly accurate standalone dense baseline of 0.4376 nDCG@10 and 0.3619 MRR@10.

Next, we combine the BM25 and BGE candidates using Reciprocal Rank Fusion (RRF) [7]. Although fusion results in a lower score of 0.3547 nDCG@10 due to less adequate lexical matching, we acquire a diverse pool of 100 candidate passages per query, which are then sent to the ColBERTv2 [3] reranker.

B. MS MARCO Results

After late - interaction token - level scoring by ColBERTv2 [3] over the RRF candidates, the TriRank pipeline improves by a great margin, reaching an nDCG@10 of 0.4638 and an MRR@10 of 0.3825. Combining strong lexical matching, excellent dense semantic retrieval and careful token - level reranking is the key to push upper bounds.

C. BEIR Zero-Shot Evaluation

Zero - Shot Evaluation To evaluate how TriRank performs out of domain, we examine the pipeline over four datasets of the BEIR [5] benchmark: SciFact, ArguAna, NFCorpus and TREC - COVID. Table II shows the cross - domain effectiveness of TriRank without any task - specific tuning.

D. Ablation Studies

We perform ablation studies to provide evidence for our choices. First, we analyze the size of the pool of candidates passed from RRF [7] fusion to the ColBERTv2 [3] reranker. We observe that a pool size of 50 is the best tradeoff with nDCG@10 0.4646 and an average latency of 15.37 ms/query. Pool sizes larger than 50, e. g. , 100 or 200, show a degradation in top-10 accuracy due to the injection of noise and also a drastic increase in computational overhead.

Second, we examine the RRF [7] smoothing parameter k. We note that the intermediate nDCG before ColBERT reranking peaks with $k = 10$ (0.3784), as opposed to the original choice of $k = 60$ (0.3547).

Lastly, we run the sparse and dense retrieval pipelines in parallel using threads and obtain an average time savings of roughly 60 ms/query, reducing the bottleneck prior to reranking.

VI. DISCUSSION

The results show that performance increases of hybrid methods stem primarily from complementarity between search components. BM25 [**?**] has strong precision when there's an exact match and prevents failures for searches with infrequent items or constrained keywords. Conversely, BGE - large - en - v1.5 [11] captures general intent when vocabulary mismatch stops the dense index from picking up the true concept.
Reciprocal Rank Fusion [7] helps connect these two components. Its similarity scoring means there's a penalty on density retriever's best performance, but RRF gives more diversity in candidate results. The density retriever's cross - attention based method means the choice and quality of candidates impacts results - if dense retrievers don't bring it into the top 10, there's not much the last step can do. The most obvious pitfall compared to single - stage dense retrievers is the added computation, though sparse and dense retrievals occur simultaneously to keep latency down.
However, ColBERT [3] MaxSim remains the bottleneck. There's a direct tradeoff between how large the candidate pool is and how responsive the system remains. We found that 50 is the highest size while staying near peak nDCG@10 performance for production use.

VII. CONCLUSION

TriRank is a scalable three - stage retrieval pipeline that combines coarse lexical retrieval, accurate dense retrieval and late token - level interaction reranking. By using memory - optimized, parallel implementations, the pipeline scales to trillions of documents such as MS MARCO [4] without hitting common hardware limitations. The three - stage architecture sets the upper bound on retrieval efficiency by returning a score of 0.4638 nDCG@10 in MS MARCO and showing strong performance in zero - shot retrieval across many BEIR [5] domains. Future work includes investigating a learnable routing strategy to dynamically skip the token - level reranker (ColBERT [3]) when possible, thus further reducing the computation.

REFERENCES

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.S. E. Robertson and H. Zaragoza,

[2] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira, "Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations," in *Proc. ACM SIGIR*, 2021, pp. 2356–2362.

[3] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "ColBERTv2: Effective and efficient retrieval via lightweight late interaction," in *Proc. NAACL*, 2022, pp. 3715–3734.

[4] P. Bajaj *et al.*, "MS MARCO: A human generated machine reading comprehension dataset," *arXiv preprint arXiv:1611.09268*, 2016.

[5] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *NeurIPS Datasets and Benchmarks*, 2021.

[6] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.

[7] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. ACM SIGIR*, 2009, pp. 758–759.

[8] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse lexical and expansion model for first stage ranking," in *Proc. ACM SIGIR*, 2021, pp. 2288–2292.

[9] "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009

[10] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in *Proc. EACL*, 2023, pp. 2014–2037.

[11] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese embedding," *arXiv preprint arXiv:2309.07597*, 2023.

[12] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.

[13] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pretrained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[14] R. Nogueira, Z. Jiang, R. Pradeep, and J. Lin, "Document ranking with a pretrained sequence-to-sequence model," in *Findings of EMNLP*, 2020, pp. 708–718.

[15] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.